



Student Information System (SIS) - Comprehensive Project & Architecture Documentation

This document is a comprehensive engineering report prepared to detail the core architectural structure, database-to-OOP integration decisions, and overall workflow of the Student Information System (SIS) project.

1. Introduction & Project Scope

The primary objective of this project is to design the core software structure that will manage the academic and administrative processes of a university. The developed system reliably simulates complex real-world data relationships (e.g., Faculty -> Department -> Program -> Student) within a software environment.

During the design process, foundational SOLID principles such as the "Single Responsibility Principle" were strictly observed. Each data model is isolated and maintained independently within its own class.

2. Implementation of Object-Oriented Programming (OOP) Principles

The project was designed entirely with a strict OOP architecture. The building blocks of this architecture are:

2.1. Encapsulation

Encapsulation is the primary security measure used to protect the state of an object. In all entity classes, data fields are restricted from the outside world using the `private` access modifier. For instance, direct external manipulation of a student's national ID (e.g., `student.nationalId = "123"`) is prevented. Instead, data access and modification are facilitated through a controlled structure using **Getter and Setter** methods.

2.2. Constructors and Data Integrity

In Java, during the object creation phase, it is vital to ensure the object resides in a valid state in memory. In this project, rather than using default (empty) constructors, **Parameterized Constructors** were implemented, mandating all necessary parameters for an object. This prevents the creation of objects with missing critical information (e.g., a student without an ID).

2.3. Polymorphism and the Object Class Relationship

Every class in Java implicitly inherits from the `Object` class. In this project, the `toString()` method is `Overridden` and customized for each entity class. This aims to output readable, formatted data dumps (logs) in the console rather than memory addresses.

3. Relational Database Mapping in Code

The project is not just a collection of simple classes; it brings a Relational Database Management System (RDBMS) schema to life in memory.

3.1. The UUID (Universally Unique Identifier) Approach

Instead of traditional auto-incrementing integer IDs, the `java.util.UUID` class was utilized.

- **Why UUID?** UUID is a modern standard used to eliminate collision risks in distributed systems and asynchronous data entries.
- **Foreign Key Management:** The program a student is enrolled in is maintained as a `UUID` `programId` rather than a direct `Program` object. This accurately reflects the 1:N (One-to-Many) and M:N (Many-to-Many) relationships present in the database schema.

3.2. Class (Table) Hierarchy

The system design is structured as follows:

- **Faculty:** The highest administrative unit of the system.
 - **Department:** A sub-administrative structure under a faculty (linked via `facultyId`).
 - **Program:** The educational curriculum linked to a department (linked via `departmentId`).
 - **Student & Instructor:** The core human resources associated with programs and departments.
-

4. Enum Architecture for Type Safety

Allowing users or the system to make text-based (String) inputs can lead to inconsistencies in the database (e.g., the confusion between "Fall", "FALL", "fall"). To solve this, a comprehensive **Enum** architecture was established.

Defined Enum Classes and Their Purposes:

- `Gender` (E, K): Standardization of gender data.
- `StudentStatus` (AKTIF, MEZUN, ASKI, AYRILDI): Tracking the student's status in the academic cycle.
- `DegreeLevel` (ONLISANS, LISANS...): Determining the degree levels of programs.
- `Semester` (GUZ, BAHAR, YAZ): Error-free processing of academic terms.
- `CourseType` (ZORUNLU, SECMELI, ASD): Logical distinctions in course selection processes.
- `PrerequisiteType` (ZORUNLU, ONERILEN): Determining the hierarchy of course prerequisites.
- `InstructorTitle` (DR, DOC_DR...): Categorization of academic titles.

The use of Enums provides compile-time checking, entirely eliminating potential errors (Run-time Exceptions) that could arise from raw String usage.

5. Main.java: System Simulation and Interaction

The `Main.java` class acts as the driver class where this designed architecture is tested and simulated.

5.1. Dynamic Data Structures: Why ArrayList?

Since the student capacity cannot be known in advance, the `java.util.ArrayList` class, which implements the `java.util.List` interface, was used instead of fixed-size Arrays (`[]`). This structure provides the advantage of rapid reading with $O(1)$ time complexity and dynamic resizing.

5.2. CLI (Command Line Interface) and Control Flow

User interaction within the system is achieved via a blocking I/O method using the `Scanner` class.

- The menu is enclosed in an infinite `while` loop, allowing the system to behave like a persistent application.

- A `switch-case` structure is used for state transitions, keeping cyclomatic complexity low and maximizing code readability compared to nested `if-else` blocks.
-

6. Java API Usage and Date Formats

Modern Java (Java 8 and above) libraries were actively used in the project:

- `java.time.LocalDate` : Used for fields requiring only dates (Year-Month-Day), such as `birthDate` or `hireDate` . Unlike the legacy `java.util.Date` class, it is immutable and therefore much safer.
 - `java.time.LocalDateTime` : Preferred for record creation timestamps (`createdAt`) where exact time down to seconds is critical.
-

7. Assessment for Future Extensibility

Since the current architecture is designed according to "Loose Coupling" and "High Cohesion" standards, future modules to be added to the project, such as:

- **JDBC / Hibernate Database Integrations,**
- **GUI / Web API Developments with JavaFX or Spring Boot,**
- **Extended Exception Handling**

can be easily integrated without disrupting the existing Core Classes. The project has been successfully modeled as a prototype of industry-standard software.